

Seismic-Sim

The Math, Explained From Scratch

This document assumes no prior background — not in structural engineering, not in the kind of math (matrices, eigenvalues, Fourier transforms) used to solve the problem. Every idea is introduced with a plain-English explanation and an analogy before any equation shows up. If you read start to finish, you should come away understanding not just what *mdof_response.py* computes, but why each step exists.

Companion to README.md — that document covers what each file in the project does; this one is about the ideas and numbers inside *mdof_response.py* specifically. Keep both updated as the project grows.

Part A builds up every concept you need from zero. Part B walks through the pipeline that turns a real earthquake recording into each floor's motion, reusing those concepts by name. Part C then applies the exact same ideas a second time to a smaller, independent system: the furniture riding on each floor. If a term ever feels unfamiliar while reading Part B or C, it was covered in Part A — jump back.

Part A — Foundations

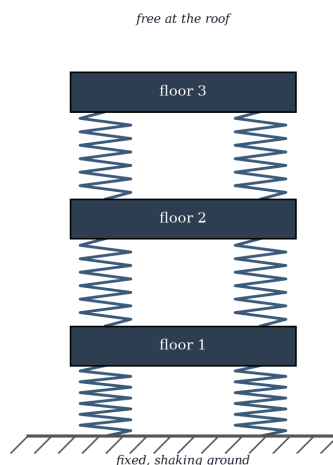
Six ideas, explained once, that everything later depends on.

A1. The big picture

Picture a real multi-story building. When the ground shakes, the building's base is forced to move with it — but the upper floors don't move in perfect lockstep. Each floor is connected to the ones above and below it by the building's structure (columns, beams), which resists being stretched or compressed, but isn't perfectly rigid.

Think of it like this: stack a few mattresses on top of each other and shove the bottom one sideways. The bottom mattress moves first; the ones above lag behind, then overshoot, compressing and stretching against their neighbors as the whole stack sways. A building does the same thing, just stiffer and faster.

This project replaces the real building with the simplest picture that captures that behavior: a stack of flat, heavy “floors,” each connected to its neighbor by something spring-like. Every floor can move sideways somewhat independently of the others. The goal of all the math that follows is: given how the ground moves, figure out exactly how far each floor moves, at every instant.



The conceptual model: floors (dark boxes) stacked on springs (zig-zag lines), sitting on the shaking ground (hatched). Fixed at the base, genuinely free at the roof — a real free-standing cantilever, not a chain propped up from above too.

A heads-up for readers of an earlier version of this document: the “spring between floors” picture above is only the intuition layer. Where that spring-like stiffness actually comes from changed in this version of the project — it used to be one abstract number, picked backward from a target sway speed. Part B2 below explains what really connects the floors now: real concrete columns and beams, sized by their own dimensions. The eigenvalue math in this Part A doesn't care which of the two produced the stiffness matrix K — it works on whatever K it's handed either way.

A2. Describing many floors at once: lists of numbers

If a building has 3 floors, and at some instant floor 1 has moved 2 mm sideways, floor 2 has moved 5 mm, and floor 3 has moved 4 mm, the most natural way to write that down is just an ordered list: (2, 5, 4) millimeters, floor 1 through 3 in order. In math, an ordered list of numbers like this is called a **vector**. Nothing more exotic than that — it's just a column of numbers where position in the list means something (here: which floor). The code stores floor positions this way constantly.

A3. Describing relationships between floors: grids of numbers

A vector can say how far each floor has moved. But to calculate the motion, we also need to describe two physical facts about the building: how heavy each floor is, and how each floor's motion pulls on its neighbors. Both are naturally described as a grid of numbers — a **matrix** — with one row and one column per floor.

The mass matrix M — “how heavy is each floor?” For a 3-floor example (each floor weighing 1,000,000 kg — 1,000 tonnes, a plausible number for a real floor plate), every floor's own weight sits on the diagonal of M (top-left to bottom-right); everywhere else is zero, because in this model one floor's weight has nothing directly to do with another floor's weight.

The stiffness matrix K — “how do the floors pull on each other?” This one is more interesting, because each connection links two floors — pulling on both whenever they move relative to each other. For a 3-floor example with a connection stiffness of about 2214.7 million newtons per meter (chosen here purely so the arithmetic below comes out to round numbers — not this project's real building, see the note below):

$$K = \begin{vmatrix} 4429.4 & -2214.7 & 0 \\ -2214.7 & 4429.4 & -2214.7 \\ 0 & -2214.7 & 2214.7 \end{vmatrix} \text{ MN/m}$$

Read row 2 (the middle row) as: “the force pulling floor 2 back toward its resting position depends on floor 2's own position (the middle number, positive — pulling it back) and on floors 1 and 3's positions too (the outer numbers, negative — a neighbor moving away drags floor 2 along with it).” Every row tells the same kind of story for its own floor.

*Look closely at the last row (floor 3, the roof): its diagonal entry is half the others, and there's no entry pulling it from “above” — because there's nothing above the roof to pull it back. This one-entry difference is what makes the model a genuine free-standing cantilever rather than a chain propped up at both ends. **This specific example (round mass, round stiffness numbers) is a made-up teaching example**, used throughout Part A purely to make matrix/eigenvalue mechanics concrete. It is not this project's real building — Part B2 works out this project's actual K from real column and beam dimensions instead, and gets a different, much larger number.*

A4. A building's favorite ways to wobble: natural frequencies and mode shapes

Push a swing and let go — it doesn't swing at just any speed you like; it settles into one particular back-and-forth rhythm determined by the swing's length. Pluck a guitar string and it vibrates in a small number of specific shapes (the fundamental note, then higher harmonics), each at its own specific pitch.

A stack of connected floors behaves the same way. Left alone to vibrate, it has a small, fixed number of natural sway shapes (one per floor), each with its own natural speed. Shake the building at exactly one of those speeds and it responds strongly in exactly that shape (resonance) — just like singing the right note can make a wine glass ring.

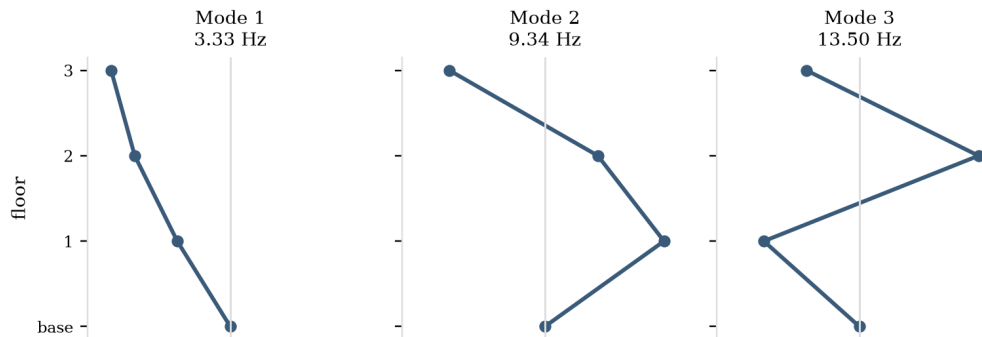
Each natural sway shape is called a **mode shape** (Greek letter ϕ , phi) — a vector, like in A2, saying how far each floor moves relative to the others when the building sways in that particular pattern. Each mode shape has a matching **natural frequency** (ω , omega) — how fast that pattern oscillates.

Finding them: the eigenvalue problem. Finding these natural shapes and speeds means finding the vectors ϕ and numbers ω that satisfy one condition: when the building is vibrating in shape ϕ , the spring-like forces trying to pull it back ($K\phi$, using the stiffness matrix from A3) must be exactly proportional to how hard inertia resists that motion ($\omega^2 M\phi$, using the mass matrix):

$$K\phi = \omega^2 M\phi$$

This kind of equation — find the special vectors and numbers that make a matrix equation like this balance — is called an **eigenvalue problem** in math (“eigen” is German for “own”/“characteristic”). ω^2 is the eigenvalue, ϕ is the matching eigenvector. This is a well-known, solved problem in math — the code hands it to a library function (`scipy.linalg.eigh`) rather than solving it by hand.

For the 3-floor example from A3, solving this gives 3 natural frequencies (3.33, 9.34, 13.50 Hz) and 3 matching mode shapes:



The 3 natural sway shapes for the example building. The fundamental mode (left) grows toward the free roof rather than forming a symmetric hump — a direct consequence of the free-top cantilever topology from A3.

A real earthquake doesn't shake a building in one clean mode shape — the actual motion is always some mix of all the modes at once. The code computes each mode's response separately (B3) and adds them back together (B3.4) — exactly the same trick as unmixing a chord into its individual notes, which is why A5 matters next.

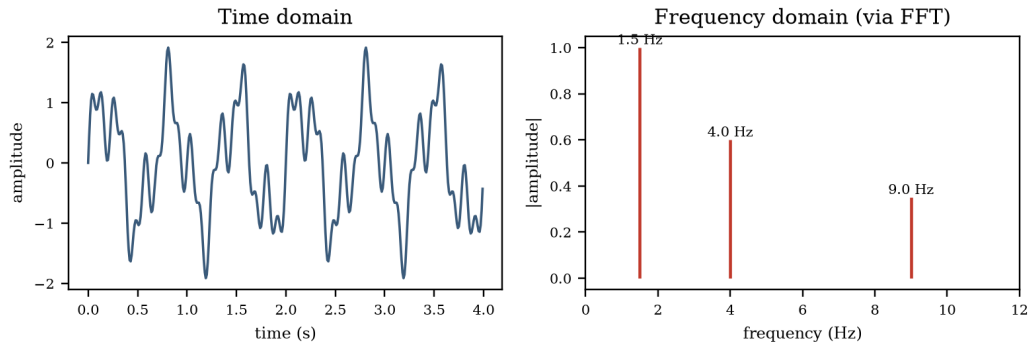
Finally, each mode gets mass-normalized to a standard size and a **participation factor** Γ (Gamma) — how strongly a uniform push on the whole building's base (like an earthquake) excites that particular mode:

$$\varphi_i^T M \varphi_i = 1 \qquad \Gamma_i = \varphi_i^T M \mathbf{1}$$

For the 3-floor example, the participation factors work out to $\Gamma_1 \approx -1656$, $\Gamma_2 \approx 474$, $\Gamma_3 \approx -182$ — all three meaningfully nonzero, because a real free-standing cantilever's mode shapes aren't symmetric.

A5. Unmixing a signal into pure tones: the Fourier transform

Any sound you hear — a chord, a voice, noise — can be broken down into a sum of simple, pure tones (single-frequency sine waves) played together at different volumes. Your ear (and a piano tuner, and an equalizer app) does this unmixing constantly. The mathematical version is called a **Fourier transform**; the fast computer algorithm that performs it is the **FFT** (Fast Fourier Transform).



The same signal, seen two ways: as a wiggly line over time (left), or as a list of “how much of each pure tone is present” (right). The FFT converts between these two views.

Two names for the same signal, from two different angles: **time domain** — the signal as most people picture it, value vs. time; **frequency domain** — the same signal described as “how much of each frequency it contains.” Why bother converting? Because some calculations that are painful in the time domain become simple multiplication in the frequency domain — integration/differentiation (B1), and “how does mode i respond to this ground motion?” (B3), both used directly in this project. A hat over a symbol, like $\hat{a}(\omega)$, means “this signal, in the frequency domain.”

A6. Why swaying dies down: damping

Push a swing once and, left alone, it doesn't swing forever — friction and air resistance gradually steal its energy until it stops. Buildings have an equivalent effect, called **damping** (ζ , zeta), usually expressed as a percentage. The code uses a fixed 5% ($\zeta = 0.05$) for every mode — a standard, generic assumption for a typical building, not measured from a specific real structure.

Part B — The Pipeline in mdof_response.py

With Part A's vocabulary in hand, here is every calculation the code actually performs to turn a real earthquake recording into each floor's motion, in order.

B1. Reading a clean ground-motion signal

B1.1 Units

PEER acceleration files are usually measured in multiples of Earth's gravity, g (9.80665 m/s²). The code converts to standard metric units. Displacement files (.DT2) are typically in centimeters and are divided by 100 to get meters.

B1.2 Recovering a missing acceleration or displacement record

The model needs both how fast the ground is accelerating (the “forcing” input) and how far it has physically moved. When a recording only supplies one of the two, the code derives the other using the frequency-domain trick from A5: repeated multiplication or division by the frequency (ω) is mathematically the same as differentiating or integrating, but far cheaper and more stable than doing it directly on the wiggly time-domain signal.

$$u_g(t) = \mathcal{F}^{-1} \left[\frac{-\hat{a}_g(\omega)}{\omega^2} \right] \quad a_g(t) = \mathcal{F}^{-1} [-\omega^2 \hat{u}_g(\omega)]$$

Dividing by ω^2 blows up as ω approaches 0, and real recordings always carry a little unwanted very-low-frequency drift/noise. Before dividing, the code passes the signal through a smooth high-pass filter (a 4th-order Butterworth filter, applied with zero phase distortion) instead of bluntly zeroing out FFT bins. The cutoff itself is adaptive: capped at 0.1 Hz (integration) / 0.05 Hz (differentiation), but never set higher than a fraction of the building's own fundamental frequency, so a very soft, long-period building's genuine low-frequency structural response doesn't get filtered out along with real drift. This is the standard “baseline correction” trick used throughout earthquake engineering to stop integrated displacement from drifting off to infinity.

B2. Building the frame-condensed stiffness model

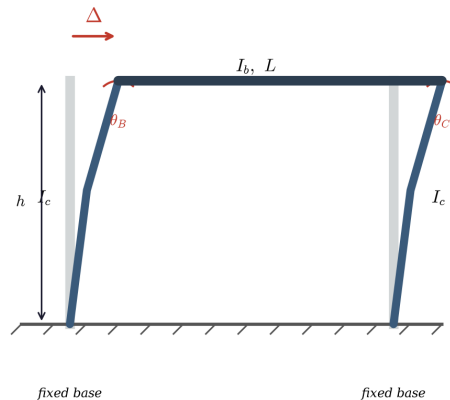
“MDOF” = multi-degree-of-freedom, from the glossary in README.md — a model with several parts (here, floors) that can each move somewhat independently. This section explains where this project's real mass matrix M and stiffness matrix K actually come from.

B2.1 Mass matrix

Exactly as in A3: every floor gets the same weight (*mass_per_floor*), so M is simply that weight repeated down the diagonal. Column and beam self-weight is neglected here — *mass_per_floor* stays the single source of floor mass, matching standard shear-building practice (a structure's own weight is typically small next to its dead/live load in this kind of idealized model).

B2.2 The building's skeleton: columns, beams, and anisotropy

In an earlier version of this project, the stiffness connecting floors was one abstract number, picked backward from a target sway speed — no real geometry anywhere in the math. This version replaces that with something real: each floor of the idealized building has four concrete columns, one at each corner, connected by beams running along all four edges — a single-bay frame, fixed into the ground at the very bottom, and completely unsupported at the very top (nothing holds the roof back except the floor below it, the same free-top cantilever idea from A1, now applied to a real frame instead of an abstract spring chain).



One story of the frame in isolation: two columns (bending stiffness I_c) of height h , fixed at the base, connected at the top by a beam (bending stiffness I_b , span L). Pushing the top sideways by Δ rotates both top joints by the same angle θ (the frame's left-right symmetry).

Each column has a rectangular cross-section: some width in the building's X direction, some (possibly different) width in the Y direction. This matters more than it sounds like it should: a column that's wide in X but thin in Y bends much more easily when the building sways in Y than when it sways in X. **This is the whole reason X and Y now get independently-built models** instead of reusing one number for both, as the old model did.

The quantity that captures “how hard is this cross-section to bend” is called the **second moment of area**, written I . For a rectangle of width b and depth d (d being the dimension in the direction the bending happens):

$$I = \frac{b d^3}{12}$$

Notice the cube on d : doubling the depth in the bending direction makes the column *eight times* stiffer in that direction, while width b only scales it linearly. That's why the column-depth sliders in the visualization have such a dramatic effect — cubes are dramatic. For a column with X-width b_x and Y-width b_y :

$$I_x = \frac{b_y b_x^3}{12} \qquad I_y = \frac{b_x b_y^3}{12}$$

Bending that resists X-direction sway uses I_x (the X-dimension is what's doing the bending, so it's the one that gets cubed); Y-direction sway is the mirror image. If $b_x = b_y$ (a square column), $I_x = I_y$ and the building would be equally stiff both ways — but the moment they differ, the two directions genuinely diverge, exactly as a real rectangular column would.

B2.3 One story in isolation: the closed-form check

Before building the full multi-story machinery, it's worth working out the simplest possible case by hand: one story of the frame, isolated, fixed at the base, free at the top, connected by one beam (the diagram above). Structural engineers have a standard tool for exactly this problem, the **slope-deflection method**: it writes down each member's end-moments as a function of the rotations at its two ends, plus a “chord rotation” term $\psi = \Delta/h$ for a column that's swayed sideways without rotating its own axis:

$$\psi = \Delta/h$$

$$\begin{array}{ll} M_{AB} = (2EI_c/h)(\theta_B - 3\psi) & M_{BA} = (2EI_c/h)(2\theta_B - 3\psi) \\ M_{DC} = (2EI_c/h)(\theta_C - 3\psi) & M_{CD} = (2EI_c/h)(2\theta_C - 3\psi) \\ M_{BC} = (2EI_b/L)(2\theta_B + \theta_C) & M_{CB} = (2EI_b/L)(2\theta_C + \theta_B) \end{array}$$

Balancing the moments at each joint (the sum of moments meeting at a joint must be zero — Newton's third law for rotation) is satisfied by the symmetric solution $\theta_B = \theta_C = \theta$ (substituting confirms both equations reduce to the same expression, and by uniqueness that's *the* solution):

$$\begin{array}{ll} \text{Joint B: } M_{BA} + M_{BC} = 0 & \text{Joint C: } M_{CD} + M_{CB} = 0 \\ \rightarrow \text{symmetric solution } \theta_B = \theta_C = \theta \text{ satisfies both} \end{array}$$

Solving for θ and substituting back into the column shear force, summed over both columns, gives the story's lateral stiffness — defining the beam-to-column stiffness ratio ρ (rho) as a clean, unit-free way to say “how much does the beam matter, relative to the column”:

$$\rho = \frac{EI_b/L}{EI_c/h} \qquad k_{story} = \frac{12EI_c}{h^3} \cdot \frac{1 + 6\rho}{2 + 3\rho}$$

Two sanity checks, both worth doing by hand — and both actually checked in [claude_scripts/verify_frame_furniture.py](#):

$$\rho \rightarrow \infty : \quad k_{story} \rightarrow \frac{24EI_c}{h^3}$$

An enormously stiff beam won't let the tops rotate at all, which is functionally the same as welding the column tops in place — exactly twice the textbook “fixed at both ends” single-column stiffness ($12EI/h^3$ per column, two columns). This is the same result the project's *old*, pre-spec-5 abstract-spring model already relied on implicitly.

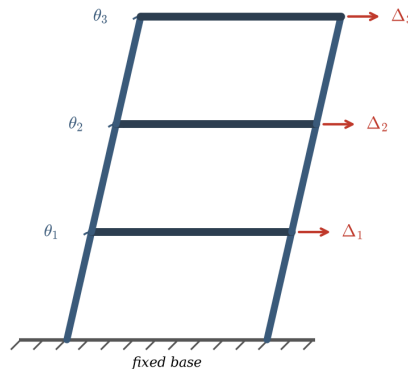
$$\rho \rightarrow 0 : \quad k_{story} \rightarrow \frac{6EI_c}{h^3}$$

With no beam at all, the two columns are just two independent free-standing flagpoles — exactly twice the stiffness of a single free-top cantilever ($3EI/h^3$ each). Both limits landing on well-known, independently-derivable answers is strong evidence the formula in between is correct too, not just plausible-looking.

Worked numbers, using this project's actual calibrated defaults ($E = 25$ GPa reinforced concrete, column depth 1.10 m in both X and Y, beam depth 1.50 m, story height $h = 3.5$ m, plan spans 8.4×6.0 m): $I_c = 0.1220 \text{ m}^4$, $I_b = 0.1406 \text{ m}^4$, giving $\rho_X \approx 0.480$ and $\rho_Y \approx 0.672$ (the two axes get different ρ only because the beam spans differ — the columns are square here). The single-story closed form gives $k_{story} \approx 963.1 \text{ MN/m}$ per frame in X, 1069.8 MN/m in Y — each frame is 2 columns, and this project has 2 identical parallel frames per axis (4 corner columns total), so the isolated-story building stiffness in this check is $2 \times$ that: $\approx 1926.1 \text{ MN/m}$ (X) / 2139.7 MN/m (Y). This exact single-story number is what `verify_frame_furniture.py` compares against the general $N=1$ assembly-and-condense code path (B2.4 below) — they match to floating-point precision.

B2.4 Generalizing to the whole building: assembly and static condensation

The single-story trick above doesn't directly generalize to a 7-story building, because story 2's columns aren't fixed at the bottom — they're attached to floor 1, which can itself rotate under load. The rigorous fix: give **every floor** two unknowns instead of one — its sideways position Δ_i (the number the rest of the project already works with) *and* its beam-column joint rotation θ_i (new). For a 7-story building that's 14 unknowns instead of 7.



Every floor gets two degrees of freedom: lateral sway Δ_i and joint rotation θ_i . Every column and beam contributes its own piece to a $2N \times 2N$ stiffness matrix before the rotations are eliminated.

Every column and every beam contributes its own little piece to a big 14×14 (in general 2N×2N) stiffness matrix, following the same slope-deflection recipe as B2.3, just written down systematically instead of by hand. Then comes the key trick, called **static condensation** (also Guyan reduction, or a Schur complement in linear-algebra terms): since nobody outside this calculation actually cares about the rotation values θ_i — the rest of the project only ever wants the N sideways positions — those rotation unknowns can be *eliminated algebraically* rather than carried around. Partition the big matrix into blocks:

$$\begin{bmatrix} K_{\Delta\Delta} & K_{\Delta\theta} \\ K_{\theta\Delta} & K_{\theta\theta} \end{bmatrix} \begin{bmatrix} \Delta \\ \theta \end{bmatrix} = \begin{bmatrix} F \\ 0 \end{bmatrix} \quad \begin{array}{l} \text{(no external moment} \\ \text{is ever applied to a joint)} \end{array}$$

From the second row, solve for θ and substitute into the first row to get the condensed relationship purely in terms of Δ :

$$\theta = -K_{\theta\theta}^{-1} K_{\theta\Delta} \Delta \quad K_{cond} = K_{\Delta\Delta} - K_{\Delta\theta} K_{\theta\theta}^{-1} K_{\theta\Delta}$$

This K_{cond} is the final N×N matrix — same shape, same meaning as the old spring-chain K — except it now comes from real geometry. The code solves $K_{\theta\theta} X = K_{\theta\Delta}$ with a linear solver (`np.linalg.solve`) rather than literally computing the matrix inverse (faster, more numerically stable), then symmetrizes the result ($\frac{1}{2}(K+K^T)$) to clean up tiny floating-point asymmetry the solve can introduce.

One subtlety: $K_{\theta\theta}^{-1}$ is a dense matrix (every entry generally nonzero), so K_{cond} ends up with some coupling between floors that aren't adjacent — floor 1 and floor 3, for instance, even though they don't touch directly. This is completely expected, physically real behavior of the condensation, not a bug — eliminating floor 2's rotation leaves a small trace of floor 2's presence connecting floors 1 and 3, and that coupling decays quickly with distance. In the rigid-beam limit it vanishes entirely and the matrix becomes exactly the familiar “only adjacent floors touch” (tridiagonal) pattern again — the same pattern the project's old spring-chain model always assumed. There are actually two identical parallel frames per axis (front and back of the building); since they're identical and don't interact, the code only ever assembles and condenses one frame, then doubles the result (`N_PARALLEL_FRAMES = 2`) — mathematically exactly equivalent to condensing both together.

For this project's default 7-story building (the calibrated defaults above), feeding the fully assembled and condensed K through the same eigenvalue machinery from A4 gives a fundamental period $T_1 \approx 1.06$ s in X and ≈ 0.95 s in Y — genuinely different between the two axes now, because the frame's real geometry, not a dialed-in target, decides it.

B2.5 Modal analysis

This is exactly the eigenvalue problem from A4, now solved with this project's real, geometry-derived M and K matrices from B2.1–B2.4, using `scipy.linalg.eigh(K, M)`. The results are sorted from lowest to highest frequency, each mode shape is mass-normalized, and each mode gets a participation factor Γ — exactly the recipe A4 already walked through, just handed a different K than the abstract teaching example used there.

B3. Solving the response in the frequency domain

B3.1 Each mode behaves like its own simple pendulum

Here's the payoff of all the setup above: instead of solving the motion of all N connected floors together (hard — they all affect each other), the modal analysis in B2.5 lets us treat each mode as if it were its own completely independent, simple back-and-forth oscillator — no interaction with the other modes at all. Mode i 's own “position” over time (its **modal coordinate**, $q_i(t)$) obeys the standard equation for a damped oscillator being pushed by an outside force — here, the ground acceleration, scaled by that mode's participation factor from B2.5:

$$\ddot{q}_i + 2\zeta\omega_{n,i}\dot{q}_i + \omega_{n,i}^2 q_i = -\Gamma_i a_g(t)$$

B3.2 Solving instantly with a frequency-response recipe

Because each mode behaves like a simple, well-understood oscillator, there's a known shortcut: compute exactly how that mode would respond to a single pure tone at any given frequency — a “recipe” function $H_i(\omega)$. This is precisely the “mix of pure tones” idea from A5:

$$H_i(\omega) = \frac{-\Gamma_i}{\omega_{n,i}^2 - \omega^2 + j2\zeta\omega_{n,i}\omega}$$

Then, because the FFT already tells us exactly which pure tones the real ground motion is made of and how loud each one is, the mode's full response is just: for every pure tone in the ground motion, multiply by how that mode reacts to a tone at that particular frequency, then add all the (now individually solved) tones back together:

$$q_i(t) = \mathcal{F}^{-1}[H_i(\omega) \hat{a}_g(\omega)]$$

Think of it like this: this is the same shortcut as knowing exactly how a wine glass rings when hit with any single pure note, then figuring out how it responds to a whole noisy room by breaking that noise into notes, looking up the glass's response to each one individually, and adding the results — instead of modeling the glass's physical vibration through the whole noisy room directly. This is done once per mode, for the entire time history at once — no step-by-step time-marching simulation (a common alternative method called Newmark- β , not used anywhere in this codebase) is needed.

B3.3 Avoiding a subtle bug: zero-padding before the FFT

fft/iff on a finite-length recording doesn't quite compute “how would this mode respond forever” — it computes a **circular convolution**, which secretly treats the recording as if it looped back to its own start the instant it ends. A lightly-damped mode's response to an early jolt can still be ringing by the end of the recording, and in a circular convolution that ringing wraps around and contaminates the very start of the computed response.

The fix is standard: pad the recording with extra zeros before the FFT (giving the ringing somewhere to decay into instead of wrapping around), then trim the padding back off after inverse-transforming. How much

padding is enough depends on how long the slowest, least-damped mode takes to die down — about 5 time constants, $1/(\zeta\omega_n)$, for that mode, capped at 4× the original length so an extremely soft, lightly-damped building (or furniture item, see Part C) can't make a single computation take unbounded time.

B3.4 Adding the modes back together

Every mode has now been solved on its own. Real floor motion is just all of them added back together, each weighted by how much that floor moves within that mode's particular shape:

$$u_{rel}(t) = \sum_i \varphi_i q_i(t)$$

This gives each floor's motion relative to the ground. Adding the ground's own motion back on top gives each floor's true, absolute position:

$$u_{abs}(t) = u_{rel}(t) + u_g(t) \mathbf{1}$$

$u_{abs}(t)$ — absolute floor position over time — is the final answer this whole pipeline exists to compute. It's what gets saved to `response_X.csv` / `response_Y.csv`. The same solve also produces each floor's *absolute acceleration* (multiplying each mode's already-computed frequency-domain response by $-\omega^2$ before the inverse FFT, rather than differentiating the finished displacement signal in the time domain) — this is what Part C's furniture needs as its own “ground motion.”

B4. From spreadsheet to animated sway

For the record you first pick, and during playback, nothing in the browser re-derives any of this physics — `index.html` only plays back the already-computed $u_{abs}(t)$ values (and Part C's furniture sway) saved to the CSV/binary files. The building is drawn as an open cutaway — the real columns, beams, and floor slabs from B2, plus furniture from Part C, all visible at once, with no exterior wall panels — so what's on screen matches what the math above actually computed, not a solid box standing in for it. Real inter-floor sway can be a fraction of a millimeter or tens of centimeters depending on the earthquake — far too small to see clearly either way — so an **amplify** slider (auto-computed per record, logarithmic range) scales the picture up purely for visibility; it never changes the underlying numbers.

B5. Live recompute: the same math, run again on demand

The Building Parameters sliders (stories, mass per floor, damping, column depth X, column depth Y, beam depth) are the one place actual physics runs outside the original offline pipeline. Moving one sends those values, plus which earthquake record is selected, to a small backend (`server.py`), which does exactly what `mdof_response.py` already does — build two fresh frame-condensed models (B2, one per axis), solve their modes (B2.5), run the same FFT-based response calculation (B3), and recompute furniture sway (Part C) — just re-invoked live, with your chosen numbers instead of the fixed defaults out/ was generated with. There is no more “Target Period” slider: once stiffness comes from real column/beam dimensions, the sway period is an *output* of the model, not something dialed in directly — a read-only T_1 readout shows what it comes out to instead.

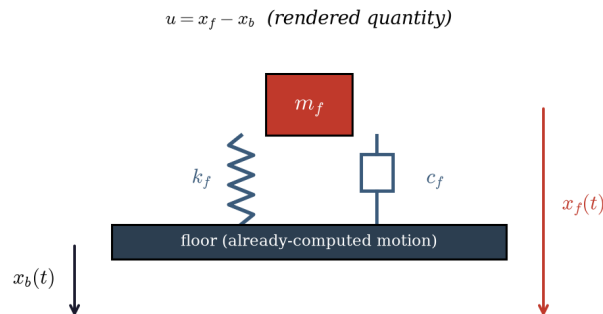
What does not get recomputed: the ground motion itself ($a_g(t)$ and $u_g(t)$) — that only depends on the earthquake recording, never on building parameters, so it's parsed and baseline-corrected once per record (B1) and cached, not redone on every slider move.

Part C — Furniture: a second, independent application of the same idea

Everything above solves for how the building itself moves. This part asks a natural next question: what about the things sitting on each floor — a table, a chair, a fan? The answer turns out to reuse every technique already introduced above, applied a second time, to a second, much smaller system.

C1. The idea: a building riding on the ground, a table riding on a floor

Each piece of furniture behaves like a tiny mass on a small spring (with a little friction, i.e. damping), riding on top of the floor's own motion — in exactly the same way the whole building rides on top of the ground's motion back in Part B. Let $x_b(t)$ be the floor's own (already computed, B3.4) absolute position and $x_f(t)$ the furniture item's absolute position. The quantity that actually gets rendered, on top of the floor's own rigid sway, is how far the furniture has moved *relative to its floor*: $u = x_f - x_b$.



A furniture item as a mass-spring-damper riding on its floor's already-computed motion — the same picture as A1's building, one level down.

$$m_f \ddot{u} + c_f \dot{u} + k_f u = -m_f \ddot{x}_b(t)$$

This is *not* an analogy — it is literally the same equation form as the building's own equation of motion under ground shaking (B3.1), with the floor standing in for the ground and the furniture item standing in for the whole building. Solving it the same way is a direct, recursive application of the same physics, not a different trick bolted on.

C2. Solving it: the transfer function

Define the furniture's own natural frequency and damping ratio, exactly as A4/A6 defined them for a whole building's mode:

$$\omega_f = \sqrt{k_f/m_f} \qquad \zeta_f = \frac{c_f}{2 m_f \omega_f}$$

The frequency-response function — “if the floor pushes at frequency ω , how much does the furniture move relative to the floor, per unit of push” — is exactly the same kind of recipe function $H_f(\omega)$ that B3.2 already introduced for the building's own modes:

$$H(j\omega) = \frac{1}{\omega_f^2 - \omega^2 + j 2\zeta_f \omega_f \omega}$$

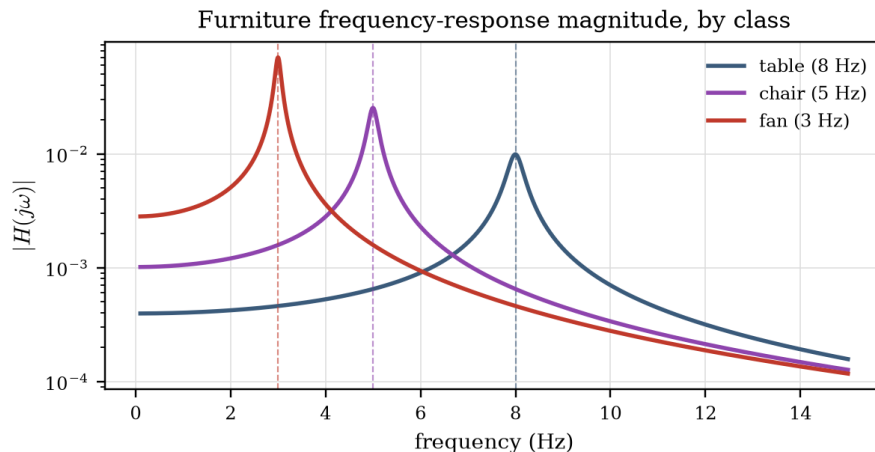
Written in Laplace form (before restricting to the imaginary axis $s = j\omega$ that the actual frequency-response evaluation uses):

$$H(s) = \frac{1}{s^2 + 2\zeta_f \omega_f s + \omega_f^2}$$

The poles of this transfer function — the values of s where the denominator hits zero — sit at:

$$s = -\zeta_f \omega_f \pm j \omega_f \sqrt{1 - \zeta_f^2}$$

directly analogous to the building's own vibration modes: a lightly-damped oscillator (small ζ_f) has poles very close to the imaginary axis, meaning it rings for a long time once excited — exactly the same qualitative behavior as a lightly-damped building mode.



$|H(j\omega)|$ for the project's three illustrative furniture classes — table (8 Hz), chair (5 Hz), fan (3 Hz), all at 2% damping. Each peaks sharply near its own natural frequency, just like a lightly-damped building mode would.

Two facts worth checking by hand (and checked automatically in `claude_scripts/verify_frame_furniture.py`): at $\omega = 0$ (a constant, unchanging push — no dynamics at all):

$$H(0) = \frac{1}{\omega_f^2}$$

Worked numbers for the table class ($f_f = 8$ Hz, $\zeta_f = 2\%$, so $\omega_f \approx 50.27$ rad/s): $H(0) = 1/\omega_f^2 \approx 3.958 \times 10^{-4}$ — just the static deflection of a mass on a spring under a constant force, no dynamics involved. The peak magnitude $|H(j\omega)|$ at $\omega \approx \omega_f$ works out to $\approx 9.89 \times 10^{-3}$ — the textbook resonance result. (The exact peak, for a damped oscillator, sits very slightly below ω_f at $\omega_f \sqrt{1-2\zeta_f^2}$ — for this project's 2% damping that's a 0.04% correction, invisible at any reasonable plotting resolution but worth naming precisely.) Both facts check out identically for the chair and fan classes at their own frequencies — only the numbers change.

C3. The actual solve

The procedure, run once per furniture class (table, chair, fan — not once per physical item, since every table on a given floor experiences identical floor motion and so has identical relative sway; only its on-screen position differs):

1. Take the floor's absolute acceleration $a_b(t)$ (B3.4's note above; see C4 for exactly how this is obtained) and Fourier-transform it, with the same zero-padding treatment B3.3 already uses — a lightly-damped furniture oscillator needs the same protection against its decaying tail wrapping around a finite-length FFT.
2. Multiply pointwise, in the frequency domain, by $-H(j\omega)$ (the minus sign is the $-m\ddot{a}_b(t)$ on the equation's right-hand side, not part of H itself):

$$U(j\omega) = -H(j\omega) \widehat{\ddot{x}_b}(\omega)$$

3. Inverse-transform back to get $u(t)$, the furniture's relative sway over time.

That pointwise multiplication in step 2 **is** the convolution theorem — multiplying two signals' Fourier transforms together is mathematically identical to convolving them in the time domain, just far cheaper to compute. It's the exact same tool B3.2 already relies on for the building's own primary solve, applied here a second time, to a second, independent, one-way-coupled system riding on top of the first. Grouping by class rather than by individual item also keeps the solve count at *floors* $\times$ 3 *classes* instead of *floors* $\times$ *item count* — for a tall building with many items per floor this is the difference between a handful of extra FFTs and hundreds.

C4. Where the floor's acceleration comes from

The building's own solve already produces each floor's *relative* displacement as a sum over modes: $q_i(t)$ for mode i . Differentiating a signal twice in the time domain (acceleration is the second derivative of position) is exactly equivalent, in the frequency domain, to multiplying its Fourier transform by $(j\omega)^2 = -\omega^2$ — a standard property of the Fourier transform, not a numerical shortcut. So: for each mode, right after computing that mode's frequency-domain response $Q_i(\omega)$, the code multiplies it by $-\omega^2$ *before* inverse-transforming, to get that mode's contribution to acceleration directly:

$$\ddot{q}_i(t) = \mathcal{F}^{-1}[-\omega^2 Q_i(\omega)]$$

This is deliberately done this way rather than numerically differentiating the finished, time-domain displacement signal afterward: that displacement signal has already been through the drift-removal filtering from B1.2, and differentiating a filtered signal a second time in the time domain would distort the result. Doing the differentiation in the frequency domain, on the un-filtered modal response, avoids that entirely — one extra inverse FFT per mode, negligible against the already-fast (tens of milliseconds) physics solve.

C5. Keeping the file size sane: decimation

A furniture response computed at the same sample rate as the ground motion (often 100–200 samples per second) would nearly double the size of out's already-substantial precomputed data, for detail nobody can actually perceive: $|H(j\omega)|$ (C2) falls off roughly as $1/\omega^2$ above the furniture's own natural frequency, meaning the furniture's response is *genuinely* band-limited — it simply doesn't contain meaningful energy at high frequencies, regardless of how finely it's sampled.

Resampling it down to roughly 50 samples per second, with a proper anti-aliasing filter (`scipy.signal.decimate`, FIR design, zero-phase so the resampled signal isn't time-shifted relative to the original), is a textbook application of the **sampling theorem**: reduce the sample rate to just above twice the signal's actual bandwidth, filter first to prevent aliasing, and nothing of substance is lost. This is why furniture data only adds a modest amount to out's size instead of doubling it — a genuine, honest application of a signals-and-systems concept, not a size fudge.

Part D — Seeing the frequency domain

Everything in Parts B and C happens in the frequency domain, but none of it was *visible*: the viewer showed a swaying building and nothing else, so the one step that does all the work — multiply, don't simulate — was invisible to anyone watching. Part D is the picture that shows it.

D1. The building's answer sheet

B3 solved one mode at a time and then added the modes up. Doing that addition *in the frequency domain instead of the time domain* collapses the whole building into a single function of frequency — its **transfer function** for floor i :

$$T_i(j\omega) = - \sum_{j=1}^N \frac{\phi_{ij} \Gamma_j}{\omega_j^2 - \omega^2 + 2j\zeta\omega_j\omega}$$

Read it as an answer sheet. For every possible frequency of shaking, it says how far floor i ends up moving. Each term in the sum is one mode's recipe H_j from B3.2, weighted by how much that floor participates in that mode's shape (ϕ_{ij}) and by how strongly the ground excites it (Γ_j). The denominator nearly vanishes when ω approaches ω_j — that is resonance, and it is why the curve has a spike at each natural frequency, with damping ζ setting how sharp the spike is.

Crucially, T_i contains no earthquake. It is built entirely from ω_j , ϕ , Γ and ζ — properties of the building. That is exactly why the drawer's middle panel changes when you drag a Building Parameter slider and its top panel never does.

D2. Input × transfer = output

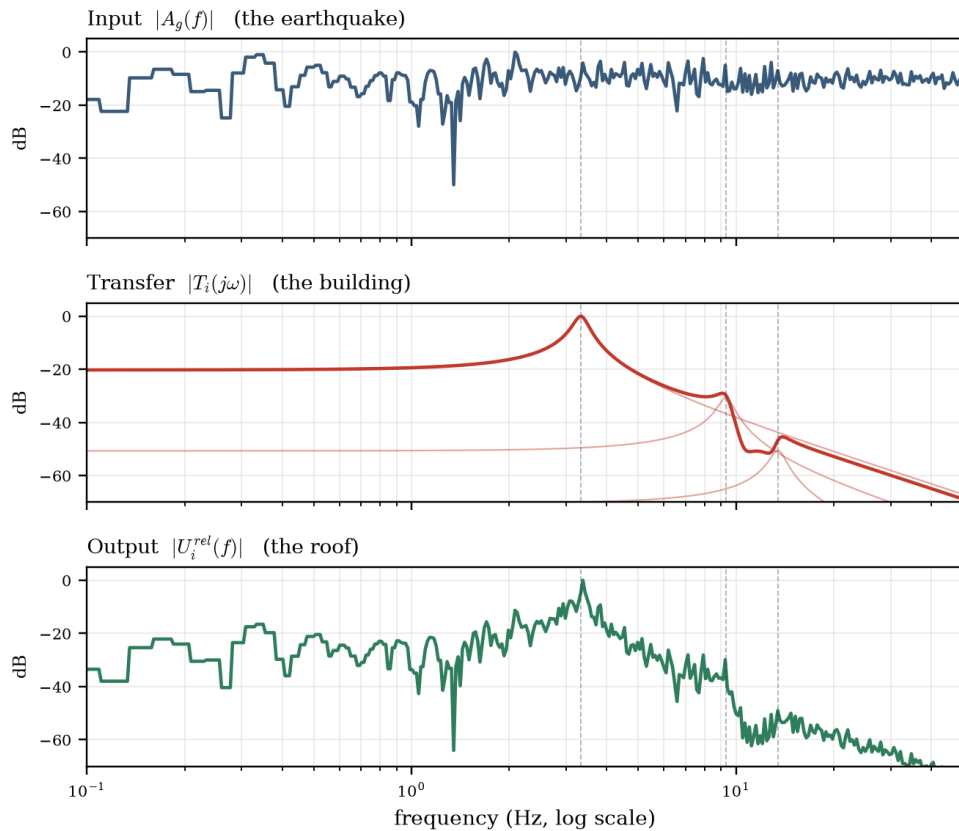
With the answer sheet in hand, the entire response is one multiplication:

$$U_i^{rel}(j\omega) = T_i(j\omega) A_g(j\omega)$$

This is the **convolution theorem** in its most useful form. In the time domain, working out the response means convolving the ground motion against the building's impulse response — an integral over all of history, for every instant. In the frequency domain the same operation is a product, frequency by frequency. Taking magnitudes gives the version you can actually plot:

$$|U_i^{rel}(f)| = |T_i(j\omega)| \cdot |A_g(f)|$$

Stack those three quantities vertically on a shared frequency axis and the multiplication becomes something you read off the page: wherever the building has a resonance *and* the earthquake has energy at that same frequency, the output has a peak. Where either one is small, the output is small. On a dB scale the product is literally a sum of heights.



Illustrative 3-floor building and a synthetic ground motion, drawn exactly as [index.html](#)'s Frequency Domain drawer draws it. Dashed lines mark the three natural frequencies; the faint curves under the Transfer panel are its individual modes. Note how the top panel's broad, featureless spread becomes a peaked output: the building has selected the frequencies it cares about out of everything the earthquake offered it.

D3. Why the output panel uses relative displacement

There is a trap here worth naming, because getting it wrong produces a plot that looks entirely reasonable. The identity above is a statement about u_{rel} — how far the floor moves *relative to the ground*, the structural deformation. But what the pipeline saves, and what the animation needs, is u_{abs} — the floor's position in space, which includes the ground's own bodily motion (B3.4).

The ground's motion is not part of the building's response to anything; it is the input, riding along. Plotting u_{abs} would mix input into output and quietly break the identity, most visibly at low frequencies where the ground's own displacement dominates. So the output panel subtracts the ground back off first. Equally: the output curve is **measured** from the computed floor motion, never obtained by multiplying the other two panels together — a picture that assumed its own conclusion would demonstrate nothing.

How exact is the identity? On the zero-padded grid the solver actually works on (B3.3), it holds to about 3×10^{-15} — floating-point noise, i.e. exact. On the trimmed arrays the browser holds it deviates slightly near the modal peaks, and the reason is B3.3 read backwards: the padding that is trimmed off contains the free-vibration ringing, which rings at precisely those frequencies. Truncating a signal is not an invertible operation in the frequency domain, so the discarded tail leaks around exactly the peaks being drawn. Log-bin averaging smooths most of it away, which is why the on-screen figure still reads correctly.

D4. Averaging into log-spaced bins

A raw FFT of a 20 000-sample record gives 10 000 frequency points spaced evenly in frequency — far more than a plot can show, and badly distributed for a log axis, where nearly all of them would pile into the right-hand third. Both the Python and JavaScript sides therefore average the magnitudes into 400 logarithmically-spaced bins between 0.1 Hz and the record's Nyquist frequency, using an identical rule so the offline figure and the browser draw the same curve. Magnitudes are scaled by dt , which makes each panel's absolute values physically meaningful and cancels out of the input/output ratio either way.

The axis stops at 50 Hz, and anything above a given record's Nyquist frequency (A5) is greyed out, because no sampled recording carries information above it. Each panel is annotated with the record's own sampling rate and Nyquist limit for the same reason.

The browser has no FFT of its own and this project has no build step, so index.html carries a hand-written iterative radix-2 Cooley-Tukey transform. It is checked against `scipy.fft` rather than trusted: agreement to $3.4\text{e-}14$, compared on complex values rather than magnitudes, since a sign-convention flip mirrors the spectrum and would be invisible in a magnitude plot.

Part E — Reshaping the earthquake itself

Every earlier part answers one question: given *this exact* recorded earthquake, how does the building respond? This part asks a different one — what if the same fault had ruptured closer, or deeper, or released more energy? Rather than inventing a fake ground motion from scratch, the model takes one of the 10 real recorded earthquakes and reshapes its own FFT spectrum with three physically-motivated multipliers. Nothing about how the building responds changes — the modal analysis, the FFT solve, the zero-padding are all unaware this happened. This part is only about how the input spectrum itself gets reshaped before any of that machinery ever sees it.

E1. Filtering, one more time

Every multiplier below is applied to $A_g(f)$ — the same ground-acceleration spectrum Part D already computes via `np.fft.rfft`. Reshaping a signal by multiplying its spectrum by some function of frequency, then inverse-transforming, is *filtering* — the same operation the adaptive baseline-correction filter (Part A) and the transfer function (Part D) both already are.

$$A'_g(f) = A_g(f) \cdot \text{scale}(f)$$

followed by `np.fft.irfft` to recover the reshaped time-domain acceleration. **scale(f)** is always real and non-negative — no phase change anywhere, the same zero-phase convention every other filter in this project uses. Two of the three effects below are frequency-*independent* (a single number, the same at every frequency); the third is a genuine per-bin filter, and that distinction is worth watching for — see E5.

E2. Magnitude — the literal Richter definition

The Richter scale isn't an arbitrary “how big” number — historically it is defined directly as an amplitude ratio:

$$M = \log_{10}(A) - \log_{10}(A_0)$$

where A is the recorded shaking amplitude and A_0 a reference amplitude. Rearranged, a magnitude difference of ΔM between two earthquakes means their amplitudes differ by exactly $10^{\Delta M}$. This project applies that definition directly, not an approximation of it:

$$\text{magnitude_scale} = 10^{M - M_0}$$

M_0 is the *real* magnitude of the selected recording (looked up per-record and baked into `out/<record>/ground_accel.json` as `reference_magnitude`), and M is wherever the Magnitude slider currently sits. At the slider's default position, $M = M_0$ and the multiplier is $10^0 = 1$ — identity, nothing about the recording changes. One whole magnitude unit above the reference multiplies every frequency bin's amplitude by exactly 10, checked directly against `rfft(reshaped) / rfft(original)`. Because this multiplier is a single number, the same at every frequency, it changes the spectrum's *height*, never its shape.

E3. Geometric spreading

An earthquake releases a fixed amount of energy at the hypocenter — the point underground where the rupture starts. As that energy radiates outward it spreads across an ever-larger sphere, the way a balloon's

surface grows as you inflate it: the same total energy spread over a bigger area means less shaking at any one point on that surface. For body waves this falls off as $1/R$, where R is the straight-line (“hypocentral”) distance from source to building — not the epicentral distance (along the ground surface only), which is why this project keeps distance and depth as two separate sliders and combines them:

$$R = \sqrt{d^2 + z^2}$$

$$R_0 = \sqrt{d_0^2 + z_0^2}$$

$$\text{spreading_scale} = R_0/R$$

d , z are the Epicenter Distance/Depth sliders’ current values; d_0 , z_0 their defaults (20 km, 10 km), matched exactly by `DEFAULT_EPICENTER_DISTANCE_KM/DEFAULT_EPICENTER_DEPTH_KM` across `mdof_response.py`, `server.py` and `index.html` so identity lands at the same point everywhere. At the defaults ($R = R_0$) this multiplier is again exactly 1. Move the synthetic hypocenter twice as far away and amplitude halves; move it to half the distance and amplitude doubles. Like magnitude, this is one flat number across every frequency — height, not shape.

E4. Anelastic attenuation — a genuine filter

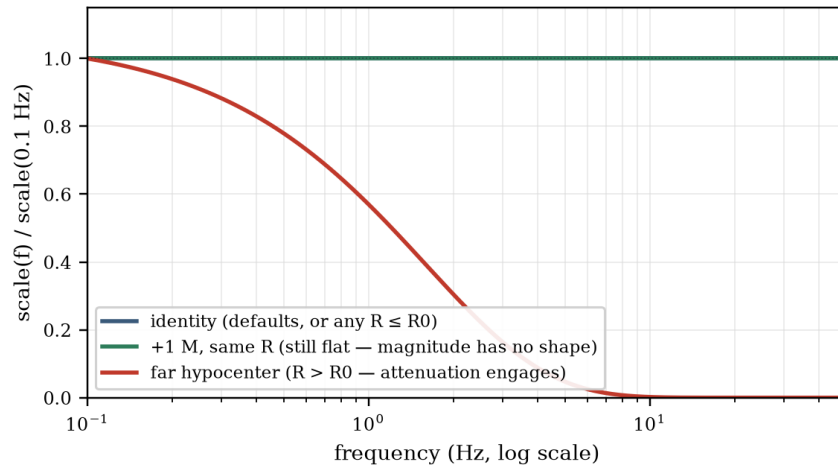
Geometric spreading alone would hold even in a perfect, lossless elastic medium — energy is only being spread out, not destroyed. Real rock isn’t lossless: a seismic wave loses some energy to heat at every point along its path, not unlike sound losing energy to air resistance, and that absorption is *frequency-dependent* — high frequencies bleed away faster than low ones. That is exactly why a distant earthquake doesn’t just feel weaker, it feels duller, its sharp jolt smoothed by the time it arrives. The frequency-dependence is what makes this a genuine filter rather than a third flat multiplier — modeling it that way, instead of one more uniform scale, is a deliberate choice to keep this project’s filtering concept legibly on display wherever it genuinely applies.

$$\text{attenuation}(f) = \exp\left(\frac{-\pi f \max(R - R_0, 0) \cdot 1000}{Q v}\right)$$

$Q = 200$ and $v = 3500$ m/s are fixed constants (typical crustal shear-wave values, not user-adjustable — same treatment as the concrete elastic modulus elsewhere in this document). The $\times 1000$ converts $(R - R_0)$ from km to meters to match v . At the defaults the exponent is exactly 0 and $\text{attenuation}(f) = 1$ everywhere — identity again, matching the other two effects.

The $\max(\cdot, 0)$ clamp is not decorative — it fixes a real bug found during this feature’s own testing.

Without it, a synthetic hypocenter *closer* than the reference recording’s own geometry makes $(R - R_0)$ negative, flipping the exponent positive: instead of decaying toward zero as frequency rises, the filter grows without bound. Run through the exponential, this doesn’t restore energy the real recording lost on its way to its instrument — that energy is simply gone, there is nothing left to restore — it amplifies whatever residual high-frequency noise the recording already carries by an unbounded factor, which at one tested combination inflated the computed floor displacement from a physically sensible number to over eleven kilometers. $R \leq R_0$ should always mean “no adjustment needed” (factor 1), never amplification — distance can only ever remove energy a real recording still has, never add back energy it never captured.



scale(f) at three parameter combinations, each curve normalized to its own value at 0.1 Hz so what's plotted is shape, not height (E2/E3 already covered height alone). Identity (blue) and a +1 M bump at the same epicenter (green) sit exactly on top of each other at 1.0 across the whole axis — magnitude changes height, never shape, so normalized they are indistinguishable. A far hypocenter (red) is the one curve that actually bends: flat near 1.0 at low frequency, rolling off toward zero as frequency climbs — the filter E4 describes, drawn rather than only asserted.

All three multiply together into the one real, non-negative scale(*f*) from E1:

$$\text{scale}(f) = \text{magnitude_scale} \cdot \text{spreading_scale} \cdot \text{attenuation}(f)$$

E5. Why the on-screen picture can still look calm

A related, separate observation from testing at extreme parameters (very close, very deep, very strong): even after the amplification bug above was fixed, dragging every slider to its most extreme combination could still make the building's sway look almost imperceptible for most of the timeline, with one single dramatic instant. This is expected, not a second bug, and follows mechanically from two already-correct pieces: a real recorded earthquake is inherently “peaky” (one dominant burst of energy, dramatically larger than the calm stretches around it — the median-to-peak ratio measured across this project's 10 records ranges from about 1:13 to 1:97), and magnitude_scale/spreading_scale are both flat multipliers that scale the whole signal — peak and calm stretches alike — by the identical factor. They cannot change how peaky the signal is relative to itself.

The on-screen sway is also compressed at render time once it grows past a record's own baseline (display $\approx$ baseline $\cdot$ (physical/baseline)^{0.35}, hard-capped at a fraction of the building's height) — a cosmetic, view-only step, added after the amplitude auto-scale described in the README's Amplify glossary entry, and after this feature's own initial verification. It exists purely so a Richter slider — where one whole step is a literal 10× jump in amplitude — cannot throw the building physically off-screen; a bigger synthetic quake still reads as visibly bigger, just not linearly so past that point. It touches only what gets drawn, never the physics above, and does not change the peakiness argument in the previous paragraph.

Symbol reference

Symbol	Plain-English meaning	Where in code
N	Number of floors being simulated	self.N
m	Weight of one floor (kg)	self.m, mass_per_floor
M, K	The mass / condensed stiffness grids of numbers (A3, B2)	self.M, self.K
ζ (zeta)	Damping — how fast sway dies out (A6)	self.zeta
ω_n (omega_n)	A mode's natural oscillation speed (A4)	self.omega_n
ϕ (phi)	A mode's natural sway shape (A4)	self.phi
Γ (Gamma)	How strongly a mode gets excited (A4, B2.5)	self.Gamma
a_g(t)	Ground acceleration over time (the input)	acceleration / self.accel
u_g(t)	Ground displacement over time	displacement / self.ground_disp
q_i(t)	Mode i's own simple back-and-forth motion	q_time[i, :]
u_rel(t)	Floor motion relative to the ground (flex only)	self.floor_disp_rel
u_abs(t)	Floor's true position in space (the final answer)	self.floor_disp_abs
a_abs(t)	Floor's absolute acceleration (feeds furniture)	self.floor_accel_abs
dt	Time between each data point in the recording (s)	dt
E	Elastic modulus of the (fixed) concrete material	E_CONCRETE
b_x, b_y	Column cross-section width in X / in Y	column_depth_x/_y
I_c, I_b	Second moment of area, column / beam cross-section	column_inertia(), beam_inertia()
h, L	Story height / beam span (per axis)	story_height, frame_span()
Δ_i , θ_i	Floor i's lateral sway / joint-rotation DOFs	assemble_frame_stiffness()
ρ (rho)	Beam-to-column stiffness ratio	frame_story_stiffness_closed_form()
k_story	One story's condensed lateral stiffness	story_stiffness
m_f, k_f, c_f	Furniture item's mass / spring / damping constant	FURNITURE_CLASSES
ω_f , ζ_f	Furniture's own natural frequency / damping ratio	furniture_frfr()
H(j ω), H(s)	Furniture (or modal) frequency-response function	furniture_frfr(), modal_H_i
T_i(j ω)	The building's transfer function for floor i (D1)	transfer_function()
A_g(f), U_i^rel(f)	Input / output magnitude spectra (D2)	log_bin_spectrum()
u(t)	Furniture's sway relative to its own floor	compute_furniture_response()
M, M0	Synthetic vs. this record's real Richter magnitude (E2)	magnitude, reference_magnitude
d, z	Epicenter distance / depth sliders (km) (E3)	epicenter_distance_km/_depth_km
R, R0	Synthetic vs. reference hypocentral distance (E3)	apply_synthetic_earthquake_scaling()
Q, v	Crustal quality factor / shear-wave speed, fixed (E4)	ATTENUATION_Q, ATTENUATION_VELOCITY_M

Symbol	Plain-English meaning	Where in code
scale(f)	Combined magnitude/spreading/attenuation filter (E1-E4)	apply_synthetic_earthquake_scaling()

Source: *mdof_response.py* (*MDOF_ShearBuilding* class, plus the module-level frame-assembly and furniture functions above it). See *README.md* for how this fits into the rest of the codebase.